

Table of Contents

TASK 1 – Environment Setup

- Configuration of User Profile
- Initialization of User Terminal

TASK 2 – Working DOCKER Images and NGC Container

- Login to the NGC Dashboard
- Generation of NGC Organization API Key
- Verification of Existing Docker Images (Before Pulling GitHub and Friend Images)
- Pulling and Executing the TensorFlow Container
- Pulling and Executing the PyTorch Container
- Pulling and Running a Use-Case-Based Container

TASK 3 – Installation and Pulling Packages from DOCKER Hub

- Pushing the Self Project Image to Docker Hub
- Pulling and Running the Self Project Image
- Pulling and Running a Friend's Docker Image
- Verifying the Docker Images List After All Pull Operations

TASK 4 – Implementing and Executing Kubernetes

- Verification of Nodes and Pods After Minikube Installation
- YAML Configuration for Pulling and Deploying the Self Project Image
- Deployment Scaling (Upscaling and Downscaling)
- Creation of Secret YAML and Deployment Using Secrets
- Verification and Execution of Secrets
- Scaling Operations on Secret-Based Deployment (Upscale & Downscale)
- Final Deployment Overview and Pod Status Verification

Oracle VirtualBox Setup and Ubuntu

Oracle VM VirtualBox is a virtualization application that allows users to operate multiple operating systems on a single computer. In this setup, Ubuntu Linux was installed within VirtualBox to establish a virtual Linux environment. This environment is used to:

- Practice operating system and command-line operations
- Create a secure and isolated testing platform

1. Installing Oracle VirtualBox

Steps:

- 1: Go to <https://www.virtualbox.org>
- 2: Download the installer for Windows hosts.
- 3: Open the installer file and click Next to begin setup.
- 4: Leave the default configuration settings and proceed with installing all required components.
- 5: Approve the installation of the network drivers when prompted.
- 6: Click Finish to complete the installation process.

2. Download Ubuntu

Steps:

- 1: Visit <https://ubuntu.com/download/desktop>
- 2: Download the Ubuntu 22.04 LTS ISO file.

3. Creating Ubuntu Virtual Machine

Steps:

- 1: Launch VirtualBox and select New.
- 2: Provide the following details:
 - Name: Ubuntu
 - Type: Linux
 - Version: Ubuntu (64-bit)
- 3: Assign memory (RAM):
 - Minimum: 2048 MB
 - Recommended: 4096 MB
- 4: Set up a virtual hard disk:
 - Disk Type: VDI (VirtualBox Disk Image)
 - Storage Allocation: Dynamically allocated
 - Disk Size: Between 20–25 GB

4. Installing Ubuntu

Steps:

- 1: Choose the created virtual machine and click Start.
- 2: Select the downloaded Ubuntu ISO file when prompted.
- 3: Click on Install Ubuntu.
- 4: Choose the Normal Installation option.
- 5: Select Erase disk and install Ubuntu (this will only modify the virtual disk, not your actual system).
- 6: Create a username and set a password.
- 7: Wait until the installation finishes, then restart the virtual machine.

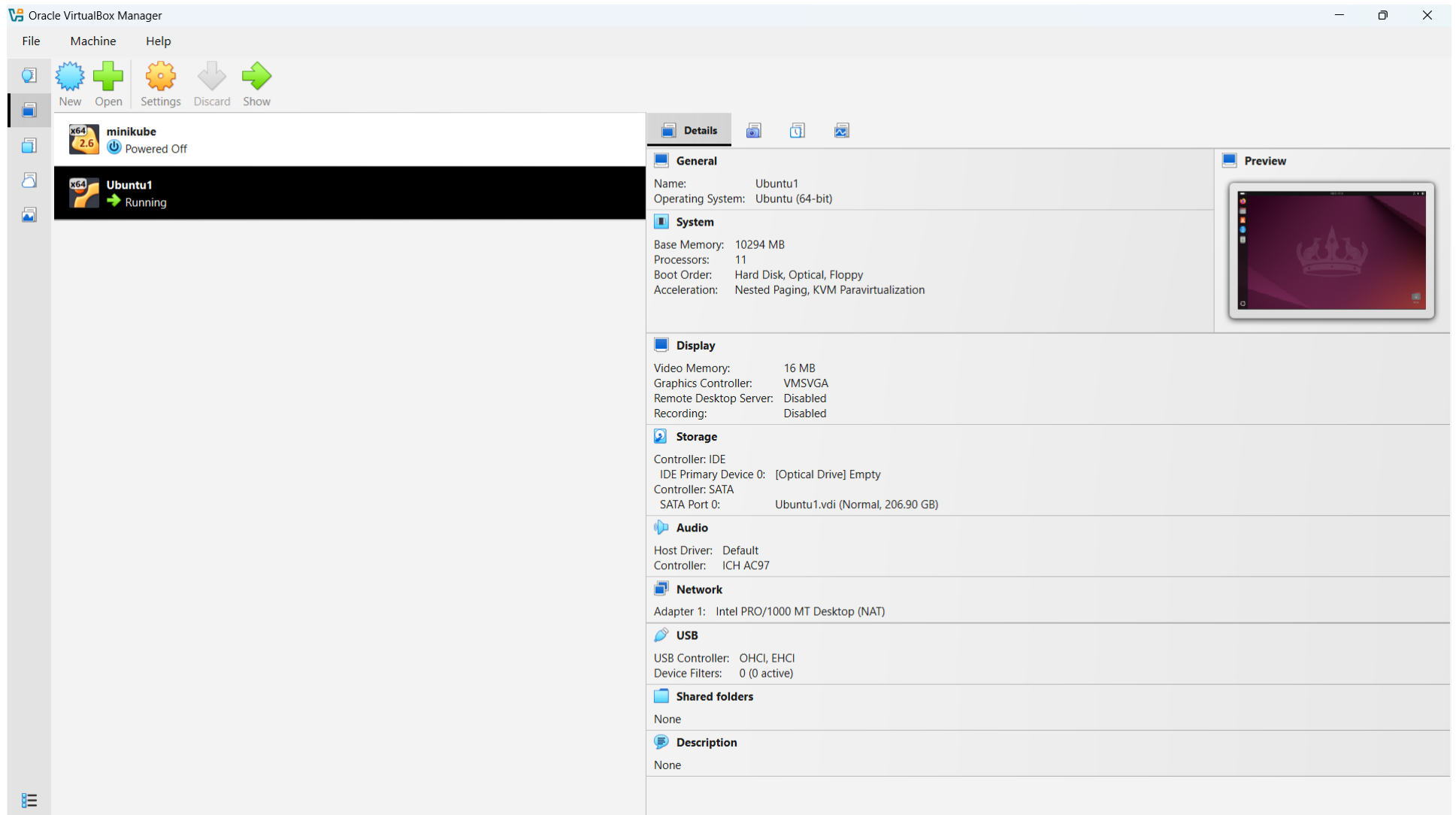


Figure 1: Ubuntu OS installed in a VM

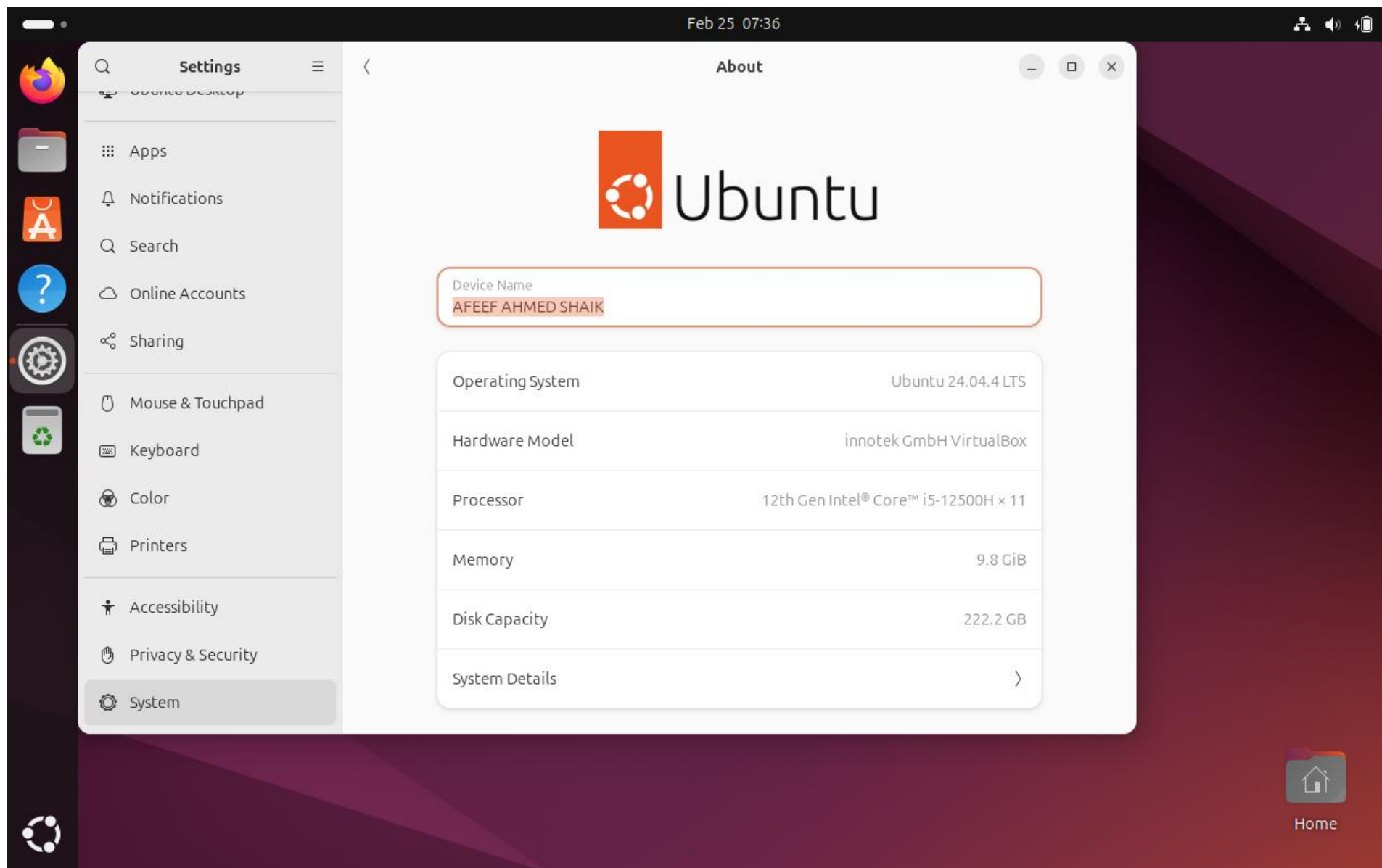


Figure 2: USER PROFILE

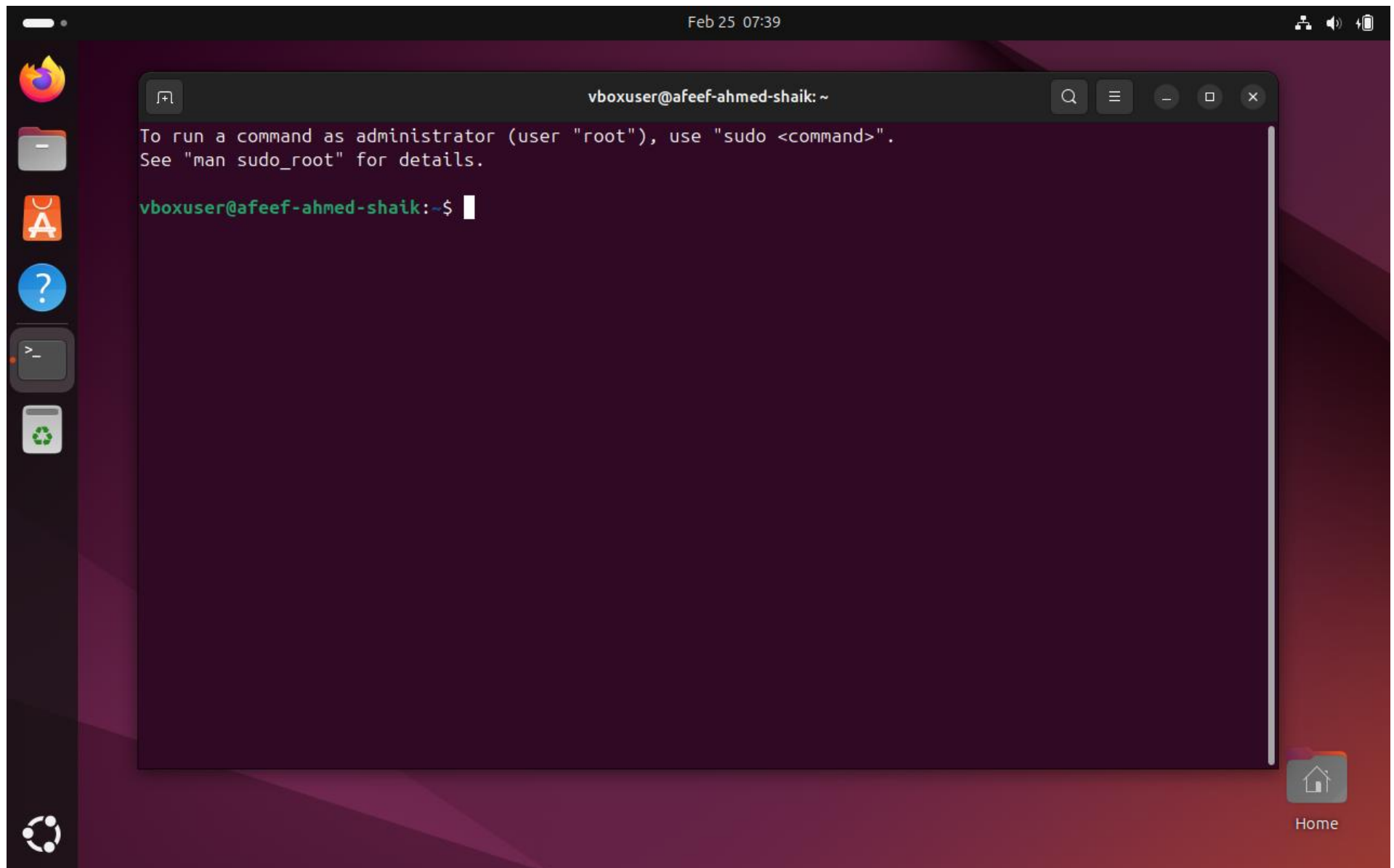


Figure3: USER TERMINAL

Task 2 : Working Docker Images and NGC Container

NVIDIA NGC Login and API Key Generation

An **NVIDIA NGC account** was created to access GPU-enabled containers available in the NVIDIA Container Registry. An API key was generated and used to authenticate Docker, allowing the system to download NGC containers securely.

Step 1: Create NVIDIA NGC Account

1. Open a web browser.
2. Go to: <https://ngc.nvidia.com>
3. Click on **Sign In**.
4. Choose **Create Account**.
5. Complete the registration form and verify your email address to activate the account.

Step 2: Generate NGC API Key

1. Sign in to the NVIDIA NGC dashboard.
2. Click on your **User Profile** icon.
3. Select **API Key** option.
4. Click on **Generate API Key**.
5. Copy the generated key and store it securely for future use.

Step 3: Login to NGC Using Docker (Ubuntu Terminal)

In the Ubuntu terminal, run the following command: Code: `docker login nvcr.io`

When prompted, enter the following credentials:

- **Username:** \$oauthtoken
- **Password:** Paste the generated NGC API key

If the login process is successful, the terminal will display:

Login Succeeded

This confirms that the system is successfully authenticated and can now pull containers from the NVIDIA NGC registry.

NGC Catalog

All you need to build AI—GPU-optimized containers, pretrained models, SDKs, and Helm charts—unified in one catalog for cloud, data-center, or edge.

Top Technology

The most-downloaded AI frameworks, models, and SDKs on NGC—curated by NVIDIA, battle-tested by the community, ready to accelerate your next project.

[All](#)
[Containers](#)
[Collections](#)
[Models](#)
[Resources](#)
[Helm Charts](#)

[View More](#)

NVIDIA PyTorch

PyTorch is a GPU accelerated tensor computational framework. Functionality can be extended with common...

High Performance Comput...

Natural Language Understand...

 Container

Updated 22 days ago

NVIDIA Triton Inference Server

Triton Inference Server is an open source software that lets teams deploy trained AI models from any framework, from...

Object Detection

Automatic Speech Recognition

 Container

Updated 20 days ago

NVIDIA cuda

Container registry for CUDA images

CUDA

 Container

Updated a month ago

NVIDIA DeepSeek-R1

NVIDIA NIM for GPU accelerated DeepSeek-R1 inference through OpenAI compatible APIs

 +1

Collection

NVIDIA Cosmos World Foundation Models

Cosmos World Foundation Models: A family of highly performant pre-trained world foundation models purpose...

 +1

Collection

NVIDIA DeepStream SDK

NVIDIA DeepStream SDK enables developers to build accelerated pipelines for a wide range of use-cases such as...

Figure 4: NVIDIA NGC USER DASHBOARD

- Dashboard
- Audit
- Organization Profile
- Teams
- Usage
- Activate Subscription
- Subscriptions
- Webhook Management

Setup > API Keys

API Keys

[+ Generate Personal Key](#)

Personal Keys

1 Result [View](#) ▾

Key ID	Key Name	Expiration	Key Value	Status	
87357941-2298-4b21-ab36-356dde07fcac	AFEED AHMED SHAIK	02/25/2027	nvapi-*****g5q	ACTIVE	⋮

25 ▾ Items

<< < 1 > >>

Go to

NGC API Keys

NVIDIA NGC API keys are required to authenticate to NGC services using NCG CLI, Docker CLI, or API communication. NVIDIA NGC supports two types of API keys.

Personal API Key

Any user who is a member of an NGC org can generate Personal API Keys. These keys are tied to the user's lifecycle within the NGC org and can access up to the permissions and services assigned to the user. During the key generation steps, users can configure which NGC services are accessible by the API key and the time-to-live from one hour to 'never expires'.

Legacy API Key

This is the original type of API key available in NGC since its inception. This type allows you to create only one 'API key' at a time. Generating a new key automatically revokes the previous one, as they cannot be rotated. The active key immediately becomes invalid when you create a new key.

Figure 5: NVIDIA NGC API ORGANIZATION DASHBOARD

Organization

Dashboard

Audit

Organization Profile

Teams

Usage

Activate Subscription

Subscriptions

Webhook Management

Setup > API Keys

API Keys

Generate Personal Key

Personal Keys

Search table

Key ID	Status
87357941-2298-4b21-ab3	ACTIVE
be3c991a-625d-4906-9cd	ACTIVE

25 Items

NGC API Keys

NVIDIA NGC API keys are required to authenticate to NGC services using NCG CLI, Docker CLI, or API communication. NVIDIA NGC supports two types of API keys.

Personal API Key

Any user who is a member of an NGC org can generate Personal API Keys. These keys are tied to the user's lifecycle within the NGC org and can access up to the permissions and services assigned to the user. During the key generation steps, users can configure which NGC services are accessible by the API key and the time-to-live from one hour to 'never expires'.

Legacy API Key

Generate Personal Key

The following Personal API Key has been successfully generated for use with this organization. This is the only time your key will be displayed.

\$ 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

Keep your Personal Key a secret. Do not share it or store it in a place where others can see or copy it.

CloseCopy Personal Key

Figure 6: NVIDIA NGC API KEY GENERATION

Docker Images Before Pulling GitHub and Friend Images

Before downloading any new Docker images, the existing images were checked to understand the current status of the Docker environment.

The following command was executed:

```
docker images
```

This command lists all Docker images stored locally on the system. It provides details such as the repository name, tag, image ID, date of creation, and image size.

By running this command, it becomes easier to verify which images are already available and to identify any newly downloaded images afterward.

At this point, only previously pulled images (for example, NVIDIA container images) were present in the system. No custom images created by the user or peers had been downloaded yet.

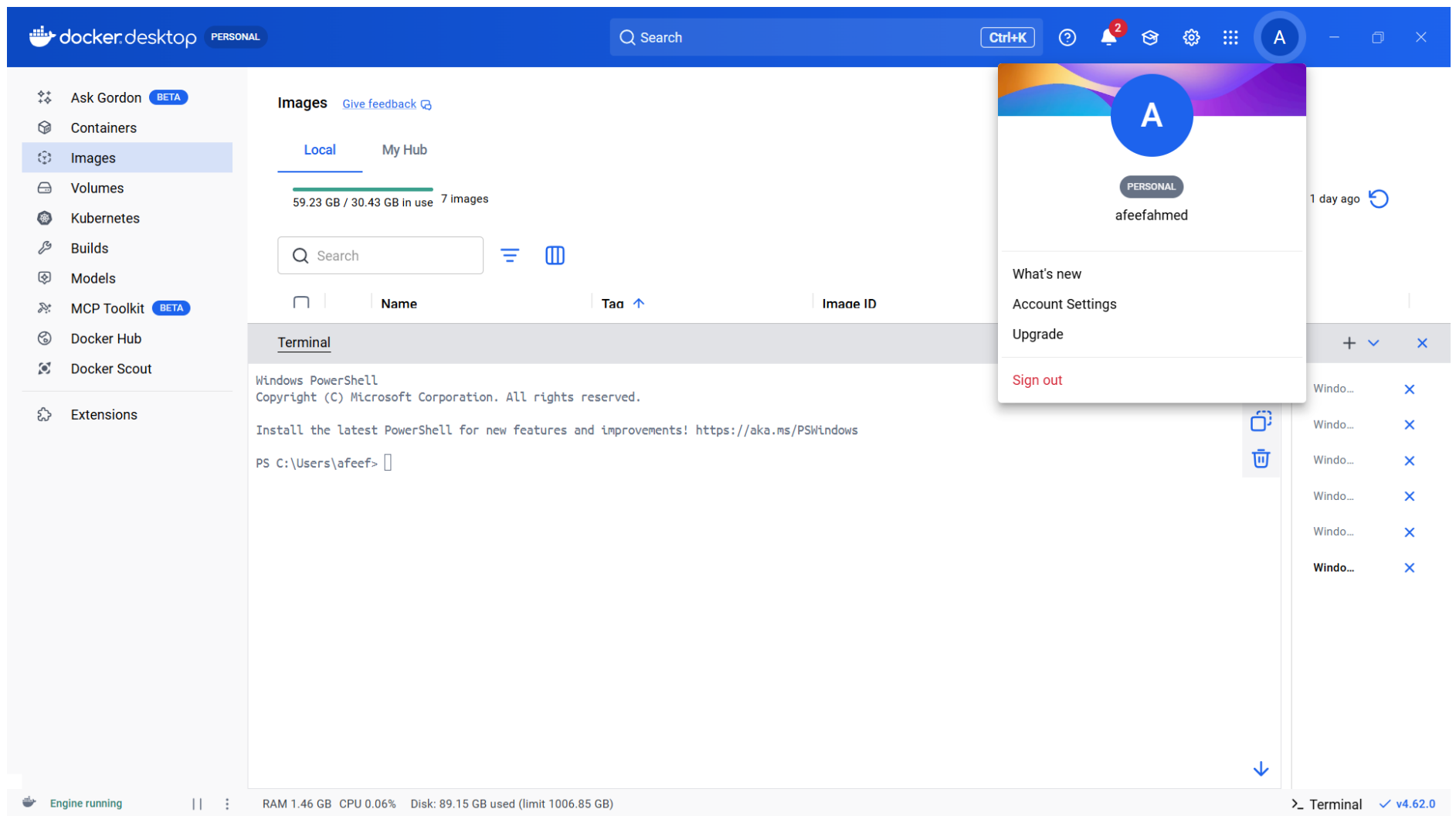


Figure 7: Docker Images Before Pulling GitHub and Friend Images

TensorFlow Container Pull & Program Execution

The TensorFlow container was obtained from the NVIDIA NGC registry to set up a pre-configured deep learning environment within Docker. This container already includes Python, TensorFlow, and all necessary libraries, removing the need for manual installation of dependencies.

```
docker pull nvcr.io/nvidia/tensorflow:25.10-tf2-py3
```

Once the download was complete, the container was launched in interactive mode using:

```
docker run -it nvcr.io/nvidia/tensorflow:25.10-tf2-py3
```

This command opened an interactive Python session inside the container.

To confirm that TensorFlow was installed properly and functioning as expected, the following Python commands were executed:

```
import tensorflow as tf
print("TensorFlow Version:", tf.__version__)
print("GPU Available:", tf.config.list_physical_devices('GPU'))
```

The output displayed the installed TensorFlow version and indicated that the container was running correctly. Since the system does not include an NVIDIA GPU, the GPU list was empty, which confirms that TensorFlow was operating in CPU mode.

This verified that the TensorFlow container was successfully downloaded and executed within the Docker environment.

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Images [Give feedback](#)

LocalMy Hub

42.12 GB / 30.43 GB in use3 Images

Last refresh: 2 hours ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	nvcv.io/nvidia/tensorflow	25.01-tf2-py3	4b15081766f8	1 year ago	31.97 GB	
☐	nvcv.io/nvidia/pytorch	26.01-py3	38ed2ecb2c16	1 month ago	30.43 GB	

Terminal

Error response from daemon: failed to resolve reference "nvcv.io/nvidia/tensorflow:26.01-tf2-py3": nvcv.io/nvidia/tensorflow:26.01-tf2-py3: not found
PS C:\Users\afeef> docker pull nvcv.io/nvidia/tensorflow:25.01-tf2-py3
25.01-tf2-py3: Pulling from nvidia/tensorflow
4f1559b21c0f: Pulling fs layer
e852726cdf4b: Pulling fs layer
a120e6e82007: Pulling fs layer
7dfa49293b99: Pulling fs layer
81355a82f4ed: Pulling fs layer
07e412f82f03: Pulling fs layer
de44b265507a: Pulling fs layer
8ec2f291c690: Pulling fs layer
2dabfd5dddb1: Pulling fs layer
4eb34e451322: Pulling fs layer
75f58769314d: Pulling fs layer
4f4fb700ef54: Pulling fs layer
5a9563dd6b7e: Pulling fs layer
4459897c0dc0: Pulling fs layer
e5a680752b35: Pulling fs layer

Engine running

RAM 7.29 GB CPU 0.00% Disk: 76.65 GB used (limit 1006.85 GB)

Terminal

v4.62.0

Figure 8: TensorFlow Container Pull

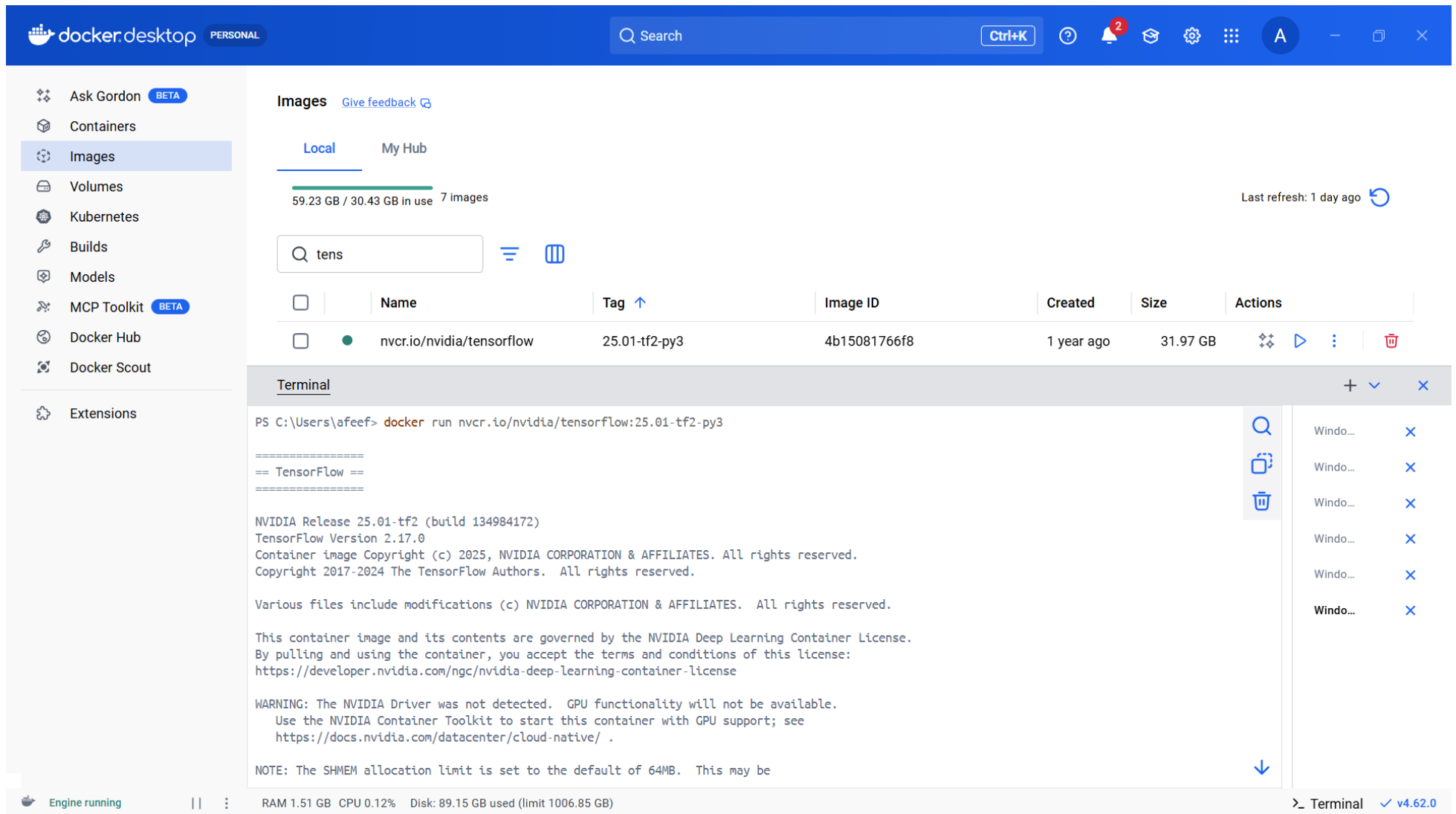


Figure 9: TensorFlow Container Run

PyTorch Container Pull & Program Execution

The PyTorch container was downloaded from the NVIDIA NGC registry to set up a ready-to-use deep learning environment within Docker. This container comes pre-installed with Python, PyTorch, CUDA libraries (if a compatible GPU is available), and all necessary dependencies, eliminating the need for manual configuration.

The image was pulled using the following command:

```
docker pull nvcr.io/nvidia/pytorch:26.10-py3
```

After the download was completed successfully, the container was launched in interactive mode with:

```
docker run -it nvcr.io/nvidia/pytorch:26.10-py3
```

This command opened a Python terminal inside the container environment.

To ensure that PyTorch was correctly installed and working properly, the following Python commands were executed:

```
import torch  
print("PyTorch Version:", torch.__version__) print("GPU Available:",  
torch.cuda.is_available())
```

The output displayed the installed PyTorch version and indicated whether GPU acceleration was available. Since the system does not have an NVIDIA GPU, the result returned **False**, confirming that PyTorch was running in CPU mode.

This verified that the PyTorch container was successfully downloaded and executed within the Docker environment.

docker desktop

PERSONAL

Q Search

Ctrl+K

2

A

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Images

[Give feedback](#)

Local

My Hub

59.23 GB / 30.43 GB in use

7 Images

Last refresh: 1 day ago

Q pyt

	Name	Tag	Image ID	Created	Size	Actions
☐	nvc	io/nvidia/pytorch	26.01-py3	38ed2ecb2c16	1 month ago	30.43 GB

Terminal

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! <https://aka.ms/PSWindows>

PS C:\Users\afeef> docker pull nvc

io/nvidia/pytorch:26.01-py3

26.01-py3: Pulling from nvidia/pytorch

Digest: sha256:38ed2ecb2c16d10677006d73fb0a150855d6ec81db8fc66e800b5ae92741007e

Status: Image is up to date for nvc

io/nvidia/pytorch:26.01-py3

nvc

io/nvidia/pytorch:26.01-py3

PS C:\Users\afeef>

+

v

x

Q

Windo...

x

Windo...

x

Windo...

x

Windo...

x

Windo...

x

Windo...

x

Engine running

RAM 1.49 GB CPU 0.06% Disk: 89.15 GB used (limit 1006.85 GB)

>_ Terminal

v4.62.0

Figure 10: PyTorch Pull

PERSONAL

Ctrl+K

?

2

A

—

×

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

59.23 GB / 30.43 GB in use 7 images

Last refresh: 1 day ago

	Name	Tag	Image ID	Created	Size	Actions
☐	nvr.cr.io/nvidia/pytorch	26.01-py3	38ed2ecb2c16	1 month ago	30.43 GB	

Terminal

+

▼

×

PS C:\Users\afeef> docker run nvr.cr.io/nvidia/pytorch:26.01-py3

=====

== PyTorch ==

=====

NVIDIA Release 26.01 (build 256811083)

PyTorch Version 2.10.0a0+a36e1d3

Container image Copyright (c) 2025, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

Copyright (c) 2014-2024 Facebook Inc.

Copyright (c) 2011-2014 Idiap Research Institute (Ronan Collobert)

Copyright (c) 2012-2014 Deepmind Technologies (Koray Kavukcuoglu)

Copyright (c) 2011-2012 NEC Laboratories America (Koray Kavukcuoglu)

Copyright (c) 2011-2013 NYU (Clement Farabet)

Copyright (c) 2006-2010 NEC Laboratories America (Ronan Collobert, Leon Bottou, Iain Melvin, Jason Weston)

Copyright (c) 2006 Idiap Research Institute (Samy Bengio)

Copyright (c) 2001-2004 Idiap Research Institute (Ronan Collobert, Samy Bengio, Johnny Mariethoz)

Copyright (c) 2015 Google Inc.

Copyright (c) 2015 Yangqing Jia

Copyright (c) 2013-2016 The Caffe contributors

All rights reserved.

🔍

📄

🗑️

Windo... ×

Windo... ×

Windo... ×

Windo... ×

Windo... ×

Windo... ×

↓

Engine running

||

:

RAM 1.54 GB CPU 0.00% Disk: 89.19 GB used (limit 1006.85 GB)

Terminal

v4.62.0

Figure 11: PyTorch Run

Use Case Container Setup Using NGC Pull and Run

The Triton Inference Server container was utilized as a practical example to demonstrate how NVIDIA NGC containers can be downloaded and executed using Docker. This container is specifically designed to deploy and serve AI models in production environments and includes optimized libraries for efficient and high-performance inference.

The container image was pulled from the NVIDIA NGC registry with the following command:

```
dockerpullnvcr.io/nvidia/tritonserver:24.10-py3
```

After the image was successfully downloaded, the container was launched to confirm that it operates correctly:

```
dockerrunnvcr.io/nvidia/tritonserver:24.10-py3
```

Once the container started, initialization logs appeared on the terminal, indicating that the Triton Inference Server was starting successfully. These logs verified that the container was functioning properly.

Triton Inference Server is widely used to deploy and manage trained machine learning models, enabling applications to send inference requests and receive real-time predictions. This experiment demonstrated how an NGC container can be effectively used for real-world AI model serving and deployment.
correctly:

docker.desktop

PERSONAL

Q

Search

Ctrl+K

2

A

—

×

⚙️ Ask Gordon BETA

📦 Containers

🖼️ Images

💾 Volumes

🌐 Kubernetes

🔑 Builds

📦 Models

🔧 MCP Toolkit BETA

🌐 Docker Hub

🔍 Docker Scout

🔧 Extensions

Images

[Give feedback](#)

Local

My Hub

59.23 GB / 30.43 GB in use

7 Images

Last refresh: 1 day ago

Q

tri

	Name	Tag	Image ID	Created	Size	Actions
☐	nvcr.io/nvidia/tritonserver	24.01-py3	338072076104	2 years ago	22.57 GB	

Terminal

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\afeef> docker pull nvcr.io/nvidia/tritonserver:24.01-py3

24.01-py3: Pulling from nvidia/tritonserver

Digest: sha256:3380720761045fc16ba3bcb96cfa54034531fc302df54ecac6b2a4deeab07bbd

Status: Image is up to date for nvcr.io/nvidia/tritonserver:24.01-py3

nvcr.io/nvidia/tritonserver:24.01-py3

PS C:\Users\afeef>

Q

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Engine running

RAM 1.45 GB CPU 0.25% Disk: 89.19 GB used (limit 1006.85 GB)

>_ Terminal

✓ v4.62.0

Figure 12: Use Case Container Pull

docker.desktop

PERSONAL

Q Search

Ctrl+K

?

2

A

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

56.97 GB / 30.43 GB in use 3 images

Last refresh: 3 hours ago

Q Search

	Name	Tag	Image ID	Created	Size	Actions
☐	nvcr.io/nvidia/tritonserver	24.01-py3	338072076104	2 years ago	22.57 GB	
☐	nvcr.io/nvidia/tensorflow	25.01-tf2-py3	4b15081766f8	1 year ago	31.97 GB	

Terminal

+ v x

PS C:\Users\afeef> docker run --rm -it -p 8000:8000 -p 8001:8001 -p 8002:8002 -v C:\triton_models:/models nvcr.io/nvidia/tritonserver:24.01-py3 tritonserver --model-repository=/models

=====

== Triton Inference Server ==

=====

NVIDIA Release 24.01 (build 80100513)

Triton Server Version 2.42.0

Copyright (c) 2018-2023, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

Various files include modifications (c) NVIDIA CORPORATION & AFFILIATES. All rights reserved.

This container image and its contents are governed by the NVIDIA Deep Learning Container License.

By pulling and using the container, you accept the terms and conditions of this license:

https://developer.nvidia.com/ngc/nvidia-deep-learning-container-license

WARNING: The NVIDIA Driver was not detected. GPU functionality will not be available.

Use the NVIDIA Container Toolkit to start this container with GPU support; see

Q

Windo... x

↓

Engine running

RAM 1.11 GB CPU 0.19% Disk: 82.10 GB used (limit 1006.85 GB)

>_ Terminal v4.62.0

Figure 13: Use Case Container Run

Task 3 : Installation and Pulling Packages from DOCKER Hub

Self Project Pushed in Docker Hub

The personal project was containerized and uploaded to Docker Hub to allow it to be shared, deployed, and executed on any system that has Docker installed.

First, a Docker image was created from the project directory using the Dockerfile with the following command:

```
build -t fitnessapp .
```

After successfully building the image, it was tagged with the Docker Hub username and version number to prepare it for uploading:

```
docker tag fitnessapp afeefahmed/fitnessapp:v1
```

Next, Docker Hub authentication was completed by running:

```
docker login
```

Once logged in, the image was pushed to the Docker Hub repository using:

```
docker push afeefahmed/fitnessapp:v1
```

During the push operation, Docker uploaded the image layers to the Docker Hub repository. A digest message confirmed that the upload was completed successfully.

This process ensures that the containerized project can be downloaded and executed on any system, providing portability and simplifying deployment.

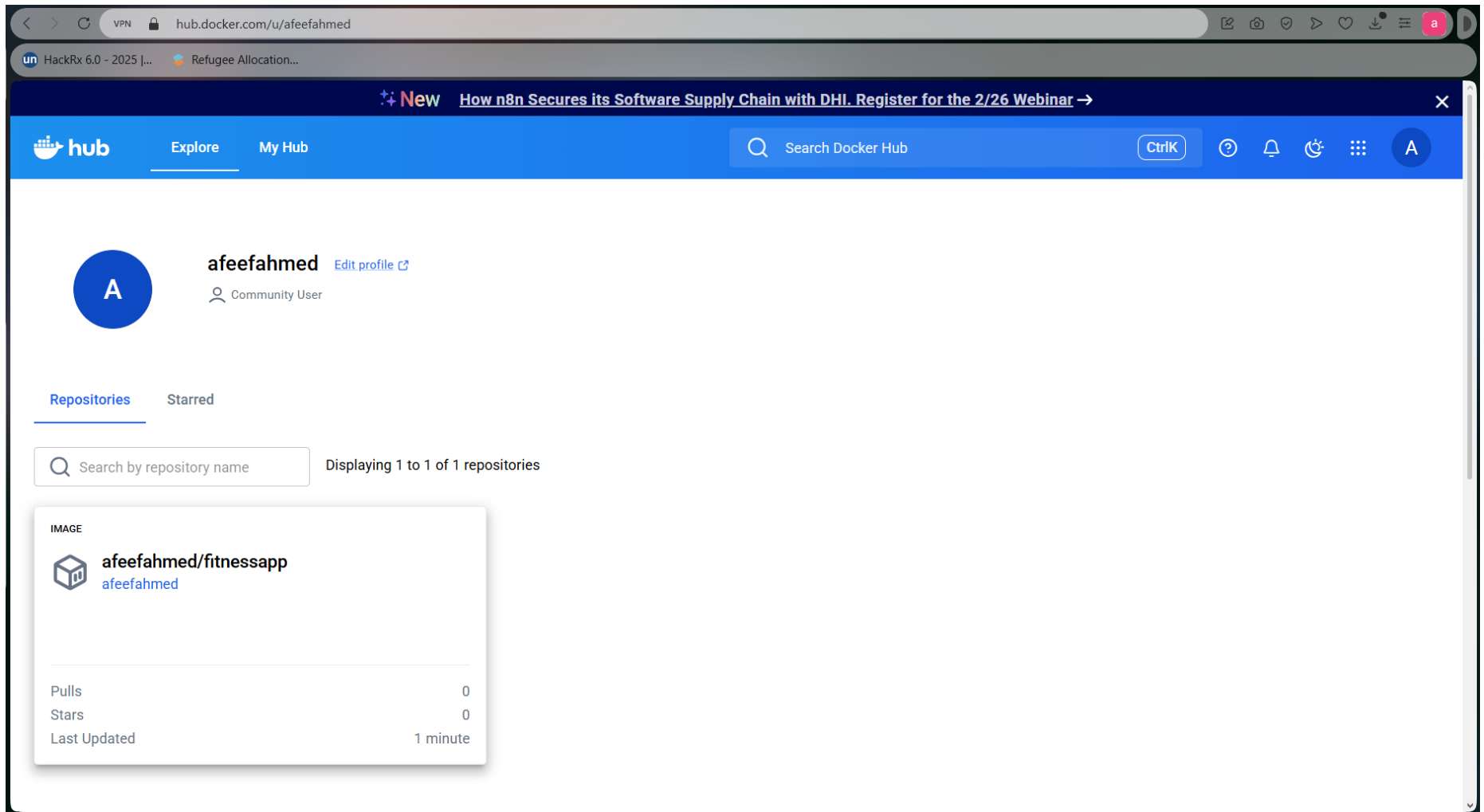


Figure 14: Self Project Pushed in Docker Hub

Pulling Self Image and Running Self Image

The project image that was previously uploaded to Docker Hub was pulled again to verify that it can be downloaded and executed on any system. Pulling the image confirms its availability in the remote repository and ensures portability across different environments.

The image was downloaded using the following command:

```
Docker pull afeefahmed/fitnessapp:v1
```

After the download was completed, Docker displayed a message indicating that the image was successfully retrieved or was already up to date.

Next, the container was launched to run the application:

```
docker run afeefahmed/:v1
```

Since the project is a web-based application, it was executed with port mapping to allow access through a web browser:

```
docker run -p 8600:8600 afeefahmed/fitnessapp:v1
```

Once the container started running, the application became accessible at:

<http://localhost:8600>

This confirmed that the project image was successfully pulled from Docker Hub and executed correctly, demonstrating container portability and simplified deployment across systems.

PERSONAL

Ctrl+K

?

2

🎓

⚙️

☰

A

—

📄

✕

🔧

Ask Gordon

BETA

📦

Containers

🔄

Images

💾

Volumes

⚙️

Kubernetes

🔧

Builds

📦

Models

🔧

MCP Toolkit

BETA

🌐

Docker Hub

🔍

Docker Scout

🔧

Extensions

Images

[Give feedback](#)

Local

My Hub

59.23 GB / 30.43 GB in use

7 Images

☰

📄

🔄

Last refresh: 1 day ago

🔄

Name	Tag	Image ID	Created	Size	Actions
Terminal Windows PowerShell Copyright (C) Microsoft Corporation. All rights reserved. Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows PS C:\Users\afeef> docker pull afeefahmed/fitnessapp:v1 v1: Pulling from afeefahmed/fitnessapp Digest: sha256:201979e3f7cf50f952191ce0feeb7bc5110d43d91c4e3e045fd2aa44459cda2d Status: Image is up to date for afeefahmed/fitnessapp:v1 docker.io/afeefahmed/fitnessapp:v1 PS C:\Users\afeef> 🔍 📄 🗑️ + ✓ ✕ Windo... ✕ Windo... ✕ Windo... ✕ Windo... ✕ Windo... ✕ Windo... ✕ Windo... ✕ Windo... ✕					

📦

Engine running

RAM 1.50 GB CPU 0.50% Disk: 89.19 GB used (limit 1006.85 GB)

📄

Terminal

✓ v4.62.0

Figure 15: Pulling Self Image

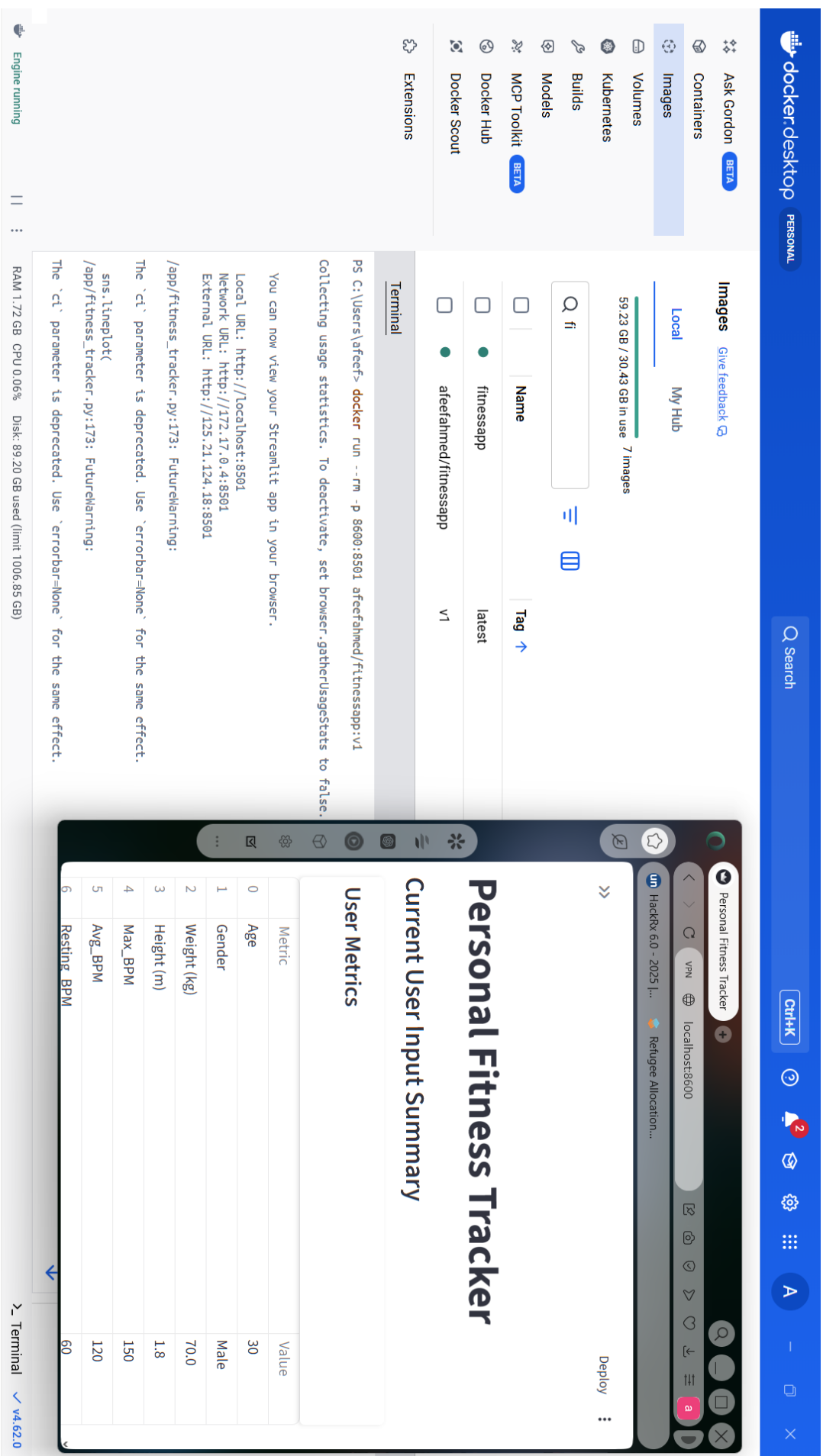


Figure 16: Running self-image

Pulling and Running Friend Image

The Docker image shared by a friend was pulled from Docker Hub to verify that containers published by other users can be successfully downloaded and executed. Pulling the image confirms that the application can be reproduced on different systems without requiring manual installation or configuration.

The image was downloaded using the following command:

```
docker pull afeefahmed/fitnessapp:v1
```

After the process completed, Docker displayed a confirmation message indicating that the image was successfully retrieved.

Next, the container was executed to launch the application:

```
docker run afeefahmed/fitnessapp:v1
```

If the application includes a web-based interface, it can be started with port mapping to allow browser access:

```
docker run -p 8600:8600 afeefahmed/fitnessapp:v1
```

Once the container began running, the application operated successfully. This demonstrated that Docker images shared by other users can be easily pulled and executed, highlighting container portability and collaboration through Docker Hub.

PERSONAL

Search

Ctrl+K

?

2

A

—

×

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

42.12 GB / 30.43 GB in use

7 images

Last refresh: 13 hours ago

Search

		Name	Tag	Image ID	Created	Size	Actions
☐	☐	python	3.10	2d8de314e8ce	20 days ago	1.6 GB	
☐	☐	websiteapp	latest	cf8fb10c9b34	9 hours ago	315.86 MB	
☐	☐	fitnessapp	latest	201979e3f7cf	1 hour ago	2.82 GB	
☐	☐	afeefahmed/fitnessapp	v1	201979e3f7cf	1 hour ago	2.82 GB	
☐	☐	sid110/hume-rainfall-app	v1	a255a3ea7b5f	10 hours ago	2.58 GB	

Showing 8 items

Terminal

+

✓

×

Search

Windo...

×

Windo...

×

↓

PS C:\Users\afeef\fitness-tracker> docker pull sid110/hume-rainfall-app:v1

Error response from daemon: failed to resolve reference "docker.io/sid110/hume-rainfall-app:v1": failed to do request: Head "https://registry-1.05da006837fe: Pull complete

d85099f0969e: Pull complete

8cbc47ff628d: Pull complete

5f6e5e6733ec: Pull complete

3a3db849ea4a: Pull complete

786b7f3b9fc7: Pull complete

64faa99400e1: Pull complete

Digest: sha256:a255a3ea7b5f9ad2bd61d8a299990f2586f1911caddf307922798200f22c49a3

Status: Downloaded newer image for sid110/hume-rainfall-app:v1

docker.io/sid110/hume-rainfall-app:v1

Engine running

RAM 1.73 GB CPU 0.13% Disk: 89.15 GB used (limit 1006.85 GB)

Terminal

v4.62.0

Figure 17: Pulling Friend Image

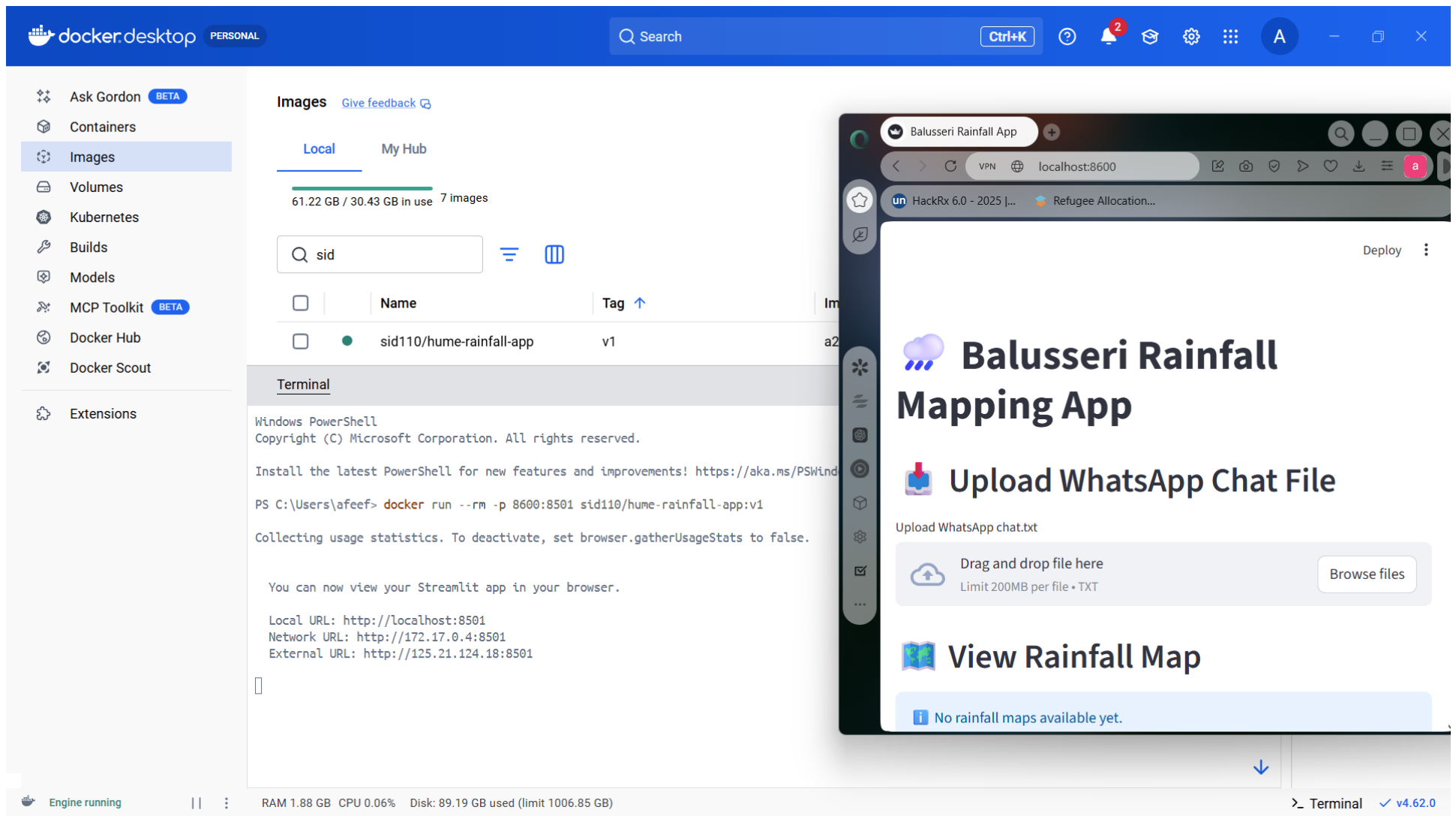


Figure 18: Running Friend Image

Docker Images After All Pulls

After downloading all the required containers, the list of Docker images was checked to ensure that every image was successfully pulled and stored locally. This step confirms that the TensorFlow, PyTorch, Triton Server, self project image, and friend's image are available in the system.

The following command was executed:

```
docker images
```

This command displays all locally stored Docker images along with important details such as:

- Repository name
- Tag (version)
- Image ID
- Creation date
- Image size

The output included images such as:

- NVIDIA TensorFlow container
- NVIDIA PyTorch container
- Triton Inference Server container
- Self project image from Docker Hub
- Friend's shared Docker image

This verification step ensured that:

- All required images were downloaded successfully
- The images are stored locally in the Docker environment
- The containers are ready to be executed whenever needed

This confirms the successful completion of the image pulling process and proper setup of the Docker environment.

PERSONAL

Search

Ctrl+K

?

2

A

-

×

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

42.12 GB / 30.43 GB in use

7 images

Last refresh: 13 hours ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	☐ nvr.io/nvidia/tritonserver	24.01-py3	338072076104	2 years ago	22.57 GB	
☐	☒ nvr.io/nvidia/tensorflow	25.01-tf2-py3	4b15081766f8	1 year ago	31.97 GB	
☐	☒ nvr.io/nvidia/pytorch	26.01-py3	38ed2ecb2c16	1 month ago	30.43 GB	
☐	☐ python	3.10	2d8de314e8ce	20 days ago	1.6 GB	

Terminal

+

▼

×

Info

→

U

In Use

Search

Refresh

Delete

↓

Windo...

×

Windo...

×

IMAGE

ID

DISK USAGE

CONTENT SIZE

EXTRA

afeefahmed/fitnessapp:v1

201979e3f7cf

2.82GB

779MB

fitnessapp:latest

201979e3f7cf

2.82GB

779MB

nvr.io/nvidia/pytorch:26.01-py3

38ed2ecb2c16

30.4GB

9.11GB

U

nvr.io/nvidia/tensorflow:25.01-tf2-py3

4b15081766f8

32GB

11.2GB

U

nvr.io/nvidia/tritonserver:24.01-py3

338072076104

22.6GB

7.73GB

python:3.10

2d8de314e8ce

1.6GB

424MB

sid110/hume-rainfall-app:v1

a255a3ea7b5f

2.59GB

593MB

websiteapp:latest

cf8fb10c9b34

316MB

102MB

PS C:\Users\afeef\fitness-tracker>

Engine running

RAM 1.52 GB CPU 0.00% Disk: 89.15 GB used (limit 1006.85 GB)

Terminal v4.62.0

Figure 19: Docker Images After All Pulls

Task 4 : Implementing and executing Kubernetes

CHECKING NODES & PODS AFTER INSTALLING MINIKUBE

After installing Minikube, the Kubernetes cluster was started to create a local single-node environment for deploying containerized applications.

The cluster was initiated using the command:

```
minikube start
```

This command performs the following actions:

- Creates a local Kubernetes cluster
- Uses the Docker driver to set up a virtual node
- Automatically configures kubectl to communicate with the cluster

Once the startup process was completed successfully, the cluster status was verified.

To check whether the node was running properly, the following command was executed:

```
kubectl get nodes
```

The output displayed the **minikube** node with the status **Ready**, confirming that the cluster was successfully initialized.

Next, all running pods across namespaces were checked using:

```
kubectl get pods -A
```

This command listed the essential system pods required for Kubernetes operations, including:

- kube-apiserver
- kube-controller-manager
- kube-scheduler
- Other core Kubernetes services

These verification steps confirmed that:

- Minikube was started successfully
- The Kubernetes node was in a Ready state
- All system components were functioning properly

This ensured that the local Kubernetes cluster was fully operational and ready for deploying containerized applications.

PERSONAL

Search

Ctrl+K

2

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Images

Local

My Hub

59.23 GB / 30.43 GB in use

7 images

Last refresh: 1 day ago

Search

Name

Tag ↑

Image ID ↑

Created

Size

Actions

Terminal

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\afeef> kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	22h	v1.35.0

PS C:\Users\afeef> kubectl get pods -A

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	fitness-deployment-84d64b4b84-l668l	1/1	Running	0	12m
default	fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	12m
kube-system	coredns-7d764666f9-slxzbz	1/1	Running	0	12m
kube-system	coredns-7d764666f9-z4gpf	1/1	Running	0	12m
kube-system	etcd-minikube	1/1	Running	0	22h
kube-system	kube-apiserver-minikube	1/1	Running	1 (8h ago)	22h
kube-system	kube-controller-manager-minikube	1/1	Running	0	22h
kube-system	kube-proxy-mjg9n	1/1	Running	0	22h
kube-system	kube-scheduler-minikube	1/1	Running	0	22h

PS C:\Users\afeef>

Search

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Engine running

RAM 1.62 GB

CPU 0.00%

Disk: 89.19 GB used (limit 1006.85 GB)

Terminal

v4.62.0

Figure 20: Checking Nodes & Pods After Installing

YAML DEFINING FOR SELF PULL

A Kubernetes deployment YAML file was created to deploy the self project image from Docker Hub into the Minikube cluster. This configuration file specifies the desired state of the application, including details such as the container image, number of replicas, and port settings.

When this YAML file is applied, Kubernetes automatically:

- Pulls the specified container image from Docker Hub
- Creates the required pods
- Maintains the defined number of replicas
- Ensures the application runs according to the configuration

The deployment configuration was saved in a file named:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fitness-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: fitnessapp
  template:
    metadata:
      labels:
        app: fitnessapp
    spec:
      containers:
        - name: fitness-container
          image: afeefahmed/fitnessapp:v1
          ports:
            - containerPort: 8501
```

This configuration directs Kubernetes to create a single replica of the container using the self project image available on Docker Hub.

Once the deployment is applied, Kubernetes automatically downloads the specified image (if not already present) and launches it within the cluster.

This ensures that the application runs according to the defined configuration inside the Minikube environment.

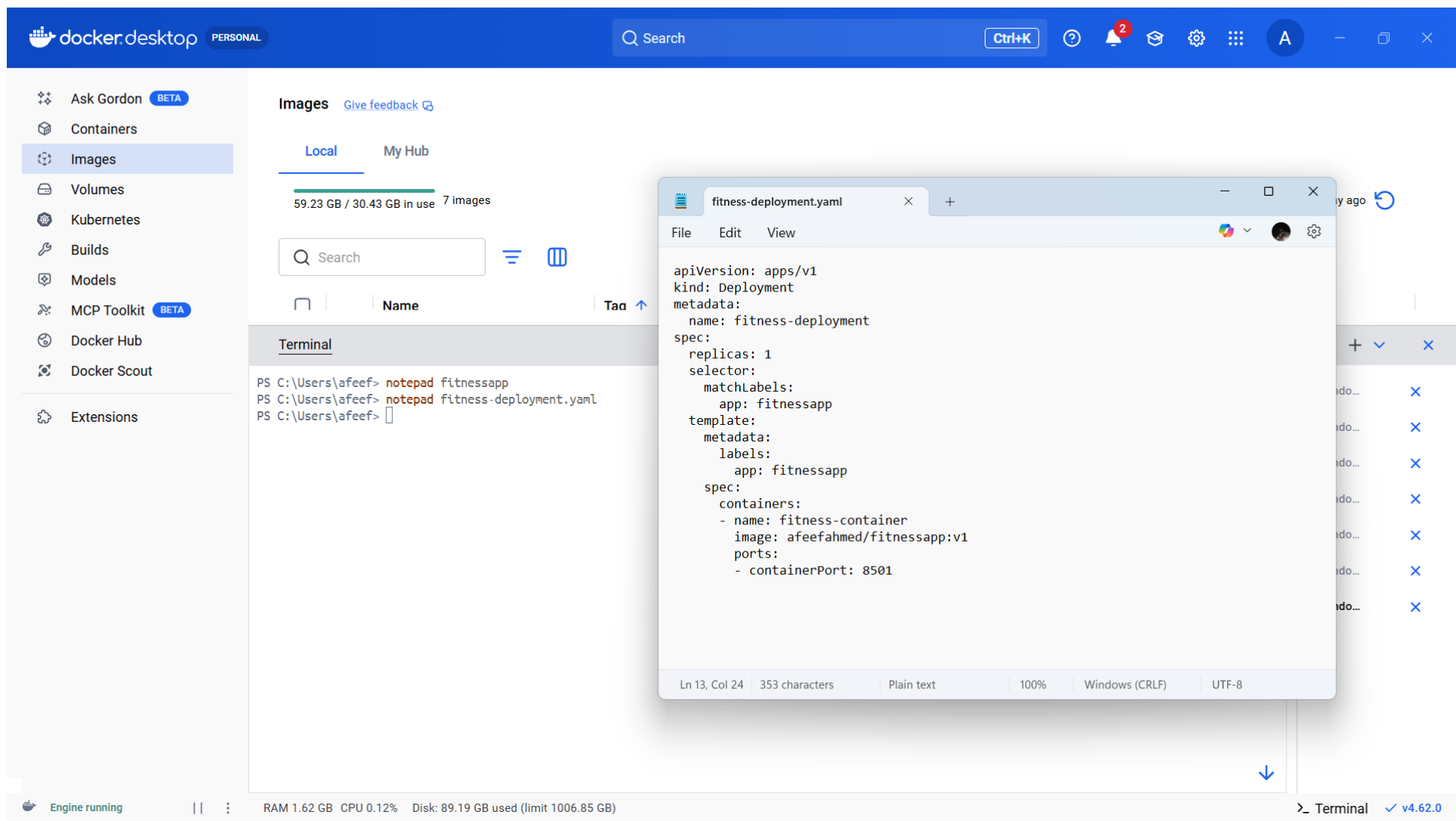


Figure 21: Yaml Defining For SelfPull

SELF PULL EXECUTION

After preparing the deployment YAML file, it was applied to the Minikube cluster to deploy the self project image. This step instructs Kubernetes to retrieve the image from Docker Hub and create the necessary pod.

The deployment was executed using the following command:

```
kubectl apply -f fitness-deployment.yaml
```

When this command was run, Kubernetes performed the following actions:

- Created the deployment resource
- Pulled the container image from Docker Hub (if not already available)
- Launched the container inside a pod

To confirm that the pod was successfully created and running, the following command was used:

```
kubectl get pods
```

The output displayed a pod associated with the deployment in the **Running** state.

This verified that the self project image was successfully pulled and executed within the Minikube Kubernetes cluster.

The screenshot displays the Docker Desktop application interface. The top navigation bar includes the Docker logo, the text "docker.desktop PERSONAL", a search bar, and a "Ctrl+K" button. The left sidebar contains a list of navigation items: Ask Gordon (BETA), Containers, Images (selected), Volumes, Kubernetes, Builds, Models, MCP Toolkit (BETA), Docker Hub, Docker Scout, and Extensions.

The main content area is divided into two sections. The top section, titled "Images", shows a progress bar for "59.23 GB / 30.43 GB in use" and "7 images". Below this is a search bar and a table with columns "Name", "Tag", and "Image ID". The bottom section, titled "Terminal", shows a Windows PowerShell session. The user has executed the following commands:

```
PS C:\Users\afeef> notepad fitness-deployment.yaml
PS C:\Users\afeef> kubectl get pods
```

The output of the `kubectl get pods` command is as follows:

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	25m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	25m

The bottom status bar indicates "Engine running", "RAM 1.66 GB", "CPU 0.06%", "Disk: 89.19 GB used (limit 1006.85 GB)", and "v4.62.0".

Overlaid on the right side of the interface is a text editor window titled "fitness-deployment.yaml". It contains the following YAML configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fitness-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: fitnessapp
  template:
    metadata:
      labels:
        app: fitnessapp
    spec:
      containers:
        - name: fitness-container
          image: afeefahmed/fitnessapp:v1
          ports:
            - containerPort: 8501
```

Figure 22: Self Pull Execution

UPSCALE

After the application was successfully deployed in Minikube, the deployment was scaled up to increase the number of active pods. Scaling enables Kubernetes to run multiple instances of the same application, which helps to:

- Handle increased user traffic
- Improve application availability
- Enhance reliability through redundancy
-

The deployment was scaled using the following command:

```
kubectrl scale deployment fitness-deployment --replicas=3
```

This command directs Kubernetes to run **three replicas** of the container instead of just one.

To confirm that the scaling process was successful, the following command was executed:

```
kubectrl get pods
```

The output showed three pods in the **Running** state, all associated with the deployment. This verified that the application was successfully scaled and multiple instances were operating within the Minikube cluster.

PERSONAL

Search

Ctrl+K

?

2

A

—

×

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

59.23 GB / 30.43 GB in use

7 images

Last refresh: 1 day ago

Search

Name

Tag

Image ID

Created

Size

Actions

Terminal

+

▼

×

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\afeef> notepad fitness-deployment.yaml

PS C:\Users\afeef> kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	32m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	32m

PS C:\Users\afeef> kubectl scale deployment fitness-deployment --replicas=3

deployment.apps/fitness-deployment scaled

PS C:\Users\afeef> kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-9qtqx	0/1	ContainerCreating	0	2s
fitness-deployment-84d64b4b84-btlhk	0/1	ContainerCreating	0	3s
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	32m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	32m

PS C:\Users\afeef>

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\afeef> notepad fitness-deployment.yaml

PS C:\Users\afeef> kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	32m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	32m

PS C:\Users\afeef> kubectl scale deployment fitness-deployment --replicas=3

deployment.apps/fitness-deployment scaled

PS C:\Users\afeef> kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-9qtqx	0/1	ContainerCreating	0	2s
fitness-deployment-84d64b4b84-btlhk	0/1	ContainerCreating	0	3s
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	32m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	32m

PS C:\Users\afeef>

Engine running

RAM 1.66 GB CPU 0.88% Disk: 89.19 GB used (limit 1006.85 GB)

Terminal v4.62.0

Figure 23: UPSCALE

DOWNSCALE

After increasing the number of replicas, the deployment was scaled down to demonstrate Kubernetes downscaling. Downscaling reduces the number of active pods and frees up unnecessary system resources.

The deployment was downscaled using the following command:

```
kubectrl scale deployment fitness-deployment --replicas=1
```

This command instructs Kubernetes to decrease the number of running replicas to **one**.

To verify that the downscaling operation was completed successfully, the following command was executed:

```
kubectrl get pods
```

The output displayed only a single pod in the **Running** state, confirming that the deployment was successfully reduced to one replica.

This demonstrated Kubernetes' ability to dynamically manage application scaling based on requirements.

PERSONAL

Ctrl+K

?

2

🎓

⚙️

☰

A

—

📄

✕

🔧 Ask Gordon BETA

📦 Containers

🔄 Images

📁 Volumes

🌐 Kubernetes

🔑 Builds

📦 Models

🔧 MCP Toolkit BETA

🌐 Docker Hub

🔍 Docker Scout

🔌 Extensions

Images [Give feedback](#)

Local

My Hub

59.23 GB / 30.43 GB in use

7 images

☰

📄

📄

Name

Tao ↑

Image ID

Created

Size

Actions

Terminal

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\afeef> kubectl scale deployment fitness-deployment --replicas=1

deployment.apps/fitness-deployment scaled

PS C:\Users\afeef> kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	34m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	34m

PS C:\Users\afeef>

🔍

📄

🗑️

Windo... ✕

Windo... ✕

Windo... ✕

Windo... ✕

Windo... ✕

Windo... ✕

Windo... ✕

Windo... ✕

Windo... ✕

Windo... ✕

🐳 Engine running

RAM 1.65 GB

CPU 0.00%

Disk: 89.19 GB used (limit 1006.85 GB)

>_ Terminal

✓ v4.62.0

Figure 24: Downscale

SECRET YAML DEFINING

A Kubernetes Secret YAML file was created to securely store configuration data within the Minikube cluster. Secrets are used to handle sensitive information such as passwords, API keys, or environment variables without embedding them directly into the application code.

The secret configuration was saved in a file named:

```
apiVersion: v1
kind: Secret
metadata:
  name: fitness-secret
type: Opaque
data:
  username: YWRtaW4=
  password: MTIzNA==
```

In this configuration:

- A Secret named **app-secret** is created
- The type **Opaque** indicates a generic secret
- A key-value pair is defined where APP_MODE is set to production
- The stringData field allows plain text values, which Kubernetes automatically converts into Base64 encoding internally

This secret can later be referenced in a deployment file, typically as an environment variable inside a container. This approach improves security by separating sensitive configuration from application code.

The screenshot displays the Docker Desktop application interface. The top navigation bar includes the Docker logo, a search bar, and various utility icons. The left sidebar contains a list of Docker components: Ask Gordon (BETA), Containers, Images (selected), Volumes, Kubernetes, Builds, Models, MCP Toolkit (BETA), Docker Hub, Docker Scout, and Extensions.

The main content area is divided into two sections. The top section, titled "Images", shows a progress bar indicating "59.23 GB / 30.43 GB in use" and "7 Images". Below this is a search bar and a table with columns for Name, Tag, and Image ID. The bottom section, titled "Terminal", shows a Windows PowerShell session. The user has executed the following commands:

```
PS C:\Users\afeef> kubectl scale deployment fitness-deployment --replicas=1
deployment.apps/fitness-deployment scaled
PS C:\Users\afeef> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	34m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	34m

```
PS C:\Users\afeef> notepad secret.yaml
PS C:\Users\afeef>
```

The "secret.yaml" file is open in a text editor, showing the following content:

```
apiVersion: v1
kind: Secret
metadata:
  name: fitness-secret
type: Opaque
data:
  username: YWRtaW4=
  password: MTIzNA==
```

The bottom status bar indicates "Engine running" and provides system resource usage: RAM 1.66 GB, CPU 0.00%, and Disk: 89.19 GB used (limit 1006.85 GB). The version "v4.62.0" is also displayed.

Figure 25: Secret Yaml Defining

NEW DEPLOYMENT DEFINING KIND SECRET

A new Kubernetes deployment configuration was created to demonstrate how a Secret can be integrated into an application through environment variables. This deployment references the previously created Secret and securely injects its value into the running container, instead of hardcoding sensitive information directly into the application code.

The configuration was saved in a file named:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fitness-secret-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: fitnesssecret
  template:
    metadata:
      labels:
        app: fitnesssecret
    spec:
      containers:
        - name: fitness-container
          image: afeefahmed/fitnessapp:v1
          ports:
            - containerPort: 8501
          env:
            - name: USERNAME
              valueFrom:
                secretKeyRef:
                  name: fitness-secret
                  key: username
            - name: PASSWORD
              valueFrom:
                secretKeyRef:
                  name: fitness-secret
                  key: password
```

Explanation of the Configuration

- A deployment named **fitness-secret-deployment** is created.
- It runs **one replica** of the container using the self project image stored on Docker Hub.
- The selector and labels ensure that Kubernetes properly identifies and manages the associated pods.
- The critical part is the **env section**, where:
 - The environment variable APP_MODE is defined.
 - Its value is not written directly in the file.
 - Instead, it is securely retrieved from the Kubernetes Secret named **app-secret**.
 - The secretKeyRef specifies both the secret name and the exact key to access.

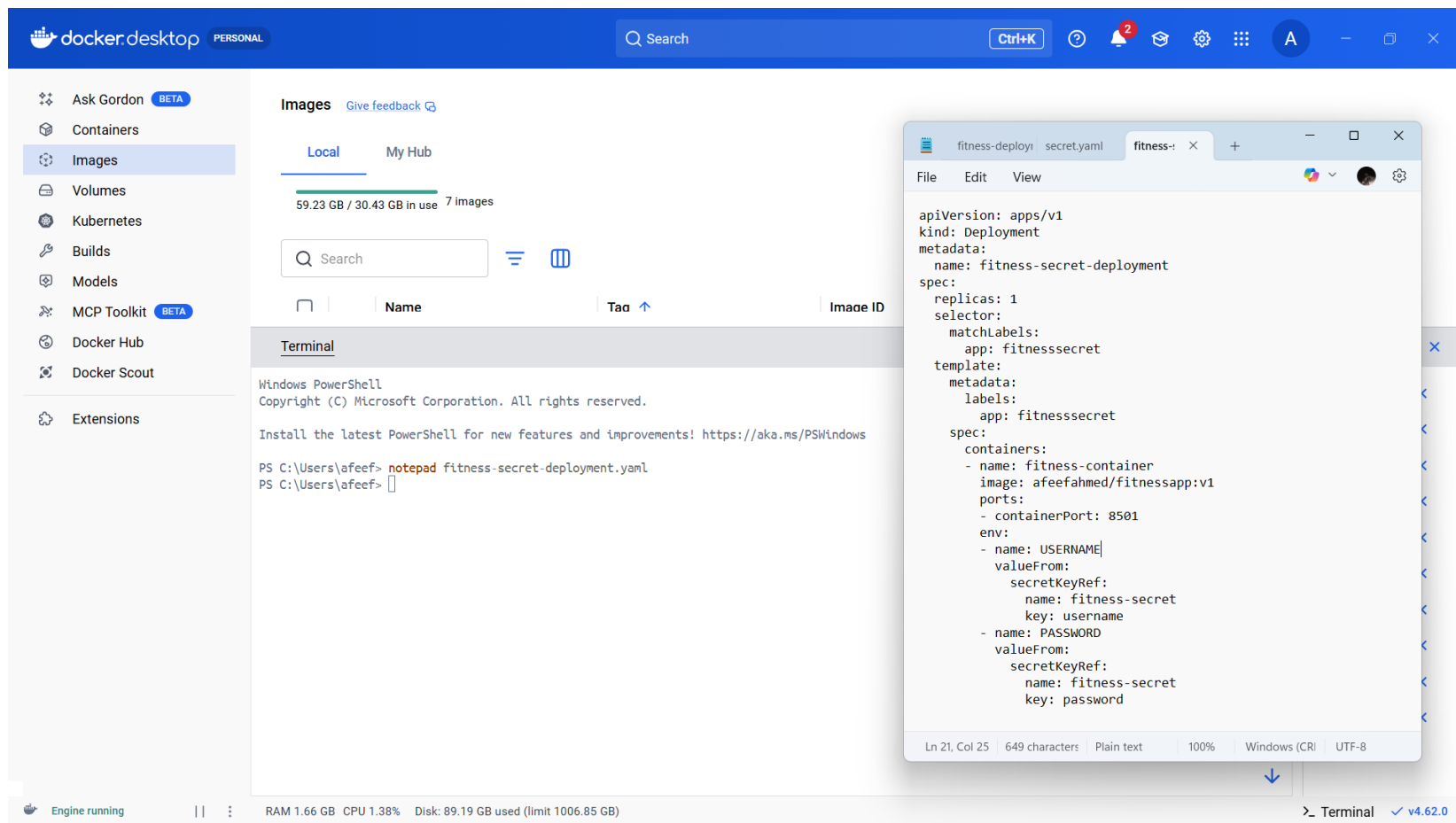


Figure 26: New Deployment Defining Kind Secret

GETTING SECRET FOR CHECK

After creating the Kubernetes Secret, it was verified to ensure that it was successfully stored in the cluster. Kubernetes provides multiple commands to check the secret's existence, metadata, and stored values.

To list all available secrets in the cluster, the following command was executed:

```
kubectl get secrets
```

This command displayed the secret named **app-secret**, confirming that it was successfully created.

To view detailed information about the secret, the following command was used:

```
kubectl describe secret app-secret
```

This command provided metadata such as:

- Secret name
- Type (Opaque)
- Available keys stored within the secret

To retrieve the complete secret information in YAML format, the following command was executed:

```
kubectl get secret app-secret -o yaml
```

The stored values appear in **Base64 encoded format** for security purposes.

To decode and view the actual value of APP_MODE, the following command was used:

```
kubectl get secret app-secret -o jsonpath="{.data.APP_MODE}" | base64 -- decode
```

This final verification step confirmed that:

- The secret was successfully created
- It was securely stored in the cluster
- Its values can be accessed by deployments when required

This demonstrates proper secret management and secure configuration handling in Kubernetes.

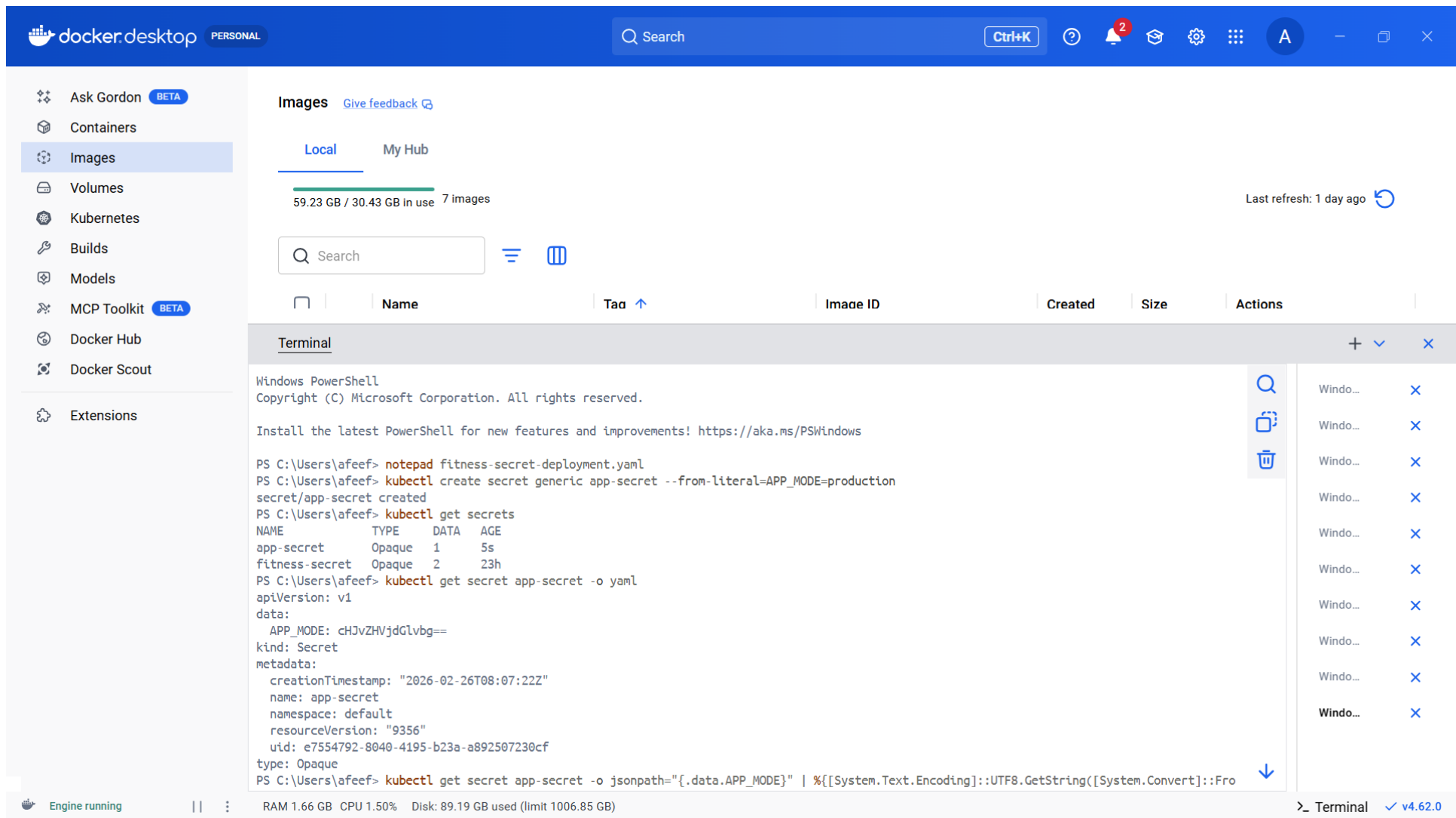


Figure 27: Getting Secret For Check

SECRET DEPLOYMENT

After creating the Kubernetes Secret, it was verified to confirm that it was properly created and stored within the cluster. Kubernetes provides several commands to inspect secret metadata and ensure its availability.

To list all existing secrets in the cluster, the following command was executed:

```
kubectl get secrets
```

The output displayed the secret named **app-secret**, confirming that it was successfully created.

To obtain detailed information about the secret, the following command was used:

```
kubectl describe secret app-secret
```

This command displayed important metadata, including:

- Secret name
- Secret type (Opaque)
- The keys stored within the secret

To retrieve the complete secret data in YAML format, the following command was executed:

```
kubectl get secret app-secret -o yaml
```

The stored values appear in **Base64 encoded format**, as Kubernetes automatically encodes secret data for secure storage.

To decode and view the actual value of the APP_MODE key, the following command was used:

```
kubectl get secret app-secret -o jsonpath="{.data.APP_MODE}" | base64 -- decode
```

This verification process confirmed that:

- The secret was successfully created
- It was securely stored within the Kubernetes cluster
- Its values can be safely accessed by deployments when required

This demonstrates proper secret management and secure handling of sensitive configuration data in Kubernetes.

PERSONAL

Ctrl+K

?

2

A

—

×

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Local

My Hub

59.23 GB / 30.43 GB in use

7 images

	Name	Tag	Image ID	Created	Size	Actions																											
Terminal secret/app-secret created PS C:\Users\afeef> kubectl get secrets <table> <thead> <tr> <th>NAME</th> <th>TYPE</th> <th>DATA</th> <th>AGE</th> </tr> </thead> <tbody> <tr> <td>app-secret</td> <td>Opaque</td> <td>1</td> <td>5s</td> </tr> <tr> <td>fitness-secret</td> <td>Opaque</td> <td>2</td> <td>23h</td> </tr> </tbody> </table> PS C:\Users\afeef> kubectl get secret app-secret -o yaml <pre> apiVersion: v1 data: APP_MODE: cHJvZHVjdGlvbg== kind: Secret metadata: creationTimestamp: "2026-02-26T08:07:22Z" name: app-secret namespace: default resourceVersion: "9356" uid: e7554792-8040-4195-b23a-a892507230cf type: Opaque </pre> PS C:\Users\afeef> kubectl get secret app-secret -o jsonpath="{.data.APP_MODE}" %[[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String(\$_))] production PS C:\Users\afeef> kubectl get pods <table> <thead> <tr> <th>NAME</th> <th>READY</th> <th>STATUS</th> <th>RESTARTS</th> <th>AGE</th> </tr> </thead> <tbody> <tr> <td>fitness-deployment-84d64b4b84-l668l</td> <td>1/1</td> <td>Running</td> <td>0</td> <td>53m</td> </tr> <tr> <td>fitness-secret-deployment-5cc7dc7fd4-6lrg5</td> <td>1/1</td> <td>Running</td> <td>0</td> <td>53m</td> </tr> </tbody> </table> PS C:\Users\afeef>							NAME	TYPE	DATA	AGE	app-secret	Opaque	1	5s	fitness-secret	Opaque	2	23h	NAME	READY	STATUS	RESTARTS	AGE	fitness-deployment-84d64b4b84-l668l	1/1	Running	0	53m	fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	53m
NAME	TYPE	DATA	AGE																														
app-secret	Opaque	1	5s																														
fitness-secret	Opaque	2	23h																														
NAME	READY	STATUS	RESTARTS	AGE																													
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	53m																													
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	53m																													

+

▼

×

🔍

📄

🗑️

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Windo...

Engine running

RAM 1.65 GB CPU 0.12% Disk: 89.19 GB used (limit 1006.85 GB)

Terminal v4.62.0

Figure 28: Secret Deployment

UPSCALE

After deploying the application with the secret configuration, the deployment was scaled up to increase the number of active pods. Upscaling improves application availability and demonstrates Kubernetes' ability to manage increased workload by running multiple instances of the same application.

The deployment was scaled using the following command:

```
kubectl scale deployment fitness-secret-deployment --replicas=3
```

This command directs Kubernetes to create **three replicas** of the pod associated with the deployment.

To confirm that the scaling process was completed successfully, the following command was executed:

```
kubectl get pods
```

The output displayed three pods in the **Running** state, all linked to **fitness-secret-deployment**.

This verification confirmed that the application was successfully upscaled and that multiple instances were running within the Kubernetes cluster, ensuring improved reliability and availability.

PERSONAL

Q Search

Ctrl+K

?

2

A

-

×

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

59.23 GB / 30.43 GB in use

7 Images

Last refresh: 1 day ago

Q Search

Name

Tao

↑

Image ID

Created

Size

Actions

Terminal

+

✓

×

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\afeef> kubectl apply -f fitness-secret-deployment.yaml

deployment.apps/fitness-secret-deployment unchanged

PS C:\Users\afeef> kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	59m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	59m

PS C:\Users\afeef> kubectl scale deployment fitness-secret-deployment --replicas=3

deployment.apps/fitness-secret-deployment scaled

PS C:\Users\afeef> kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
fitness-deployment-84d64b4b84-l668l	1/1	Running	0	60m
fitness-secret-deployment-5cc7dc7fd4-6lrg5	1/1	Running	0	60m
fitness-secret-deployment-5cc7dc7fd4-nkzbd	0/1	ContainerCreating	0	2s
fitness-secret-deployment-5cc7dc7fd4-xr576	0/1	ContainerCreating	0	2s

PS C:\Users\afeef>

RAM 1.66 GB

CPU 0.06%

Disk: 89.19 GB used (limit 1006.85 GB)

> Terminal

✓ v4.62.0

Figure 29: UPSCALE

DOWNSCALE

After demonstrating the upscaling process, the deployment was scaled down to decrease the number of active pods. Downscaling helps optimize resource usage and illustrates how Kubernetes can dynamically adjust application capacity based on requirements.

The deployment was reduced using the following command:

```
kubectrl scale deployment fitness-secret-deployment --replicas=1
```

This command instructs Kubernetes to lower the number of pod replicas to **one**.

To confirm that the downscaling process was completed successfully, the following command was executed:

```
kubectrl get pods
```

The output displayed only one pod in the **Running** state for **fitness-secret-deployment**, indicating that the additional pods were successfully terminated.

This demonstrates Kubernetes' ability to efficiently manage scaling operations while maintaining application stability.

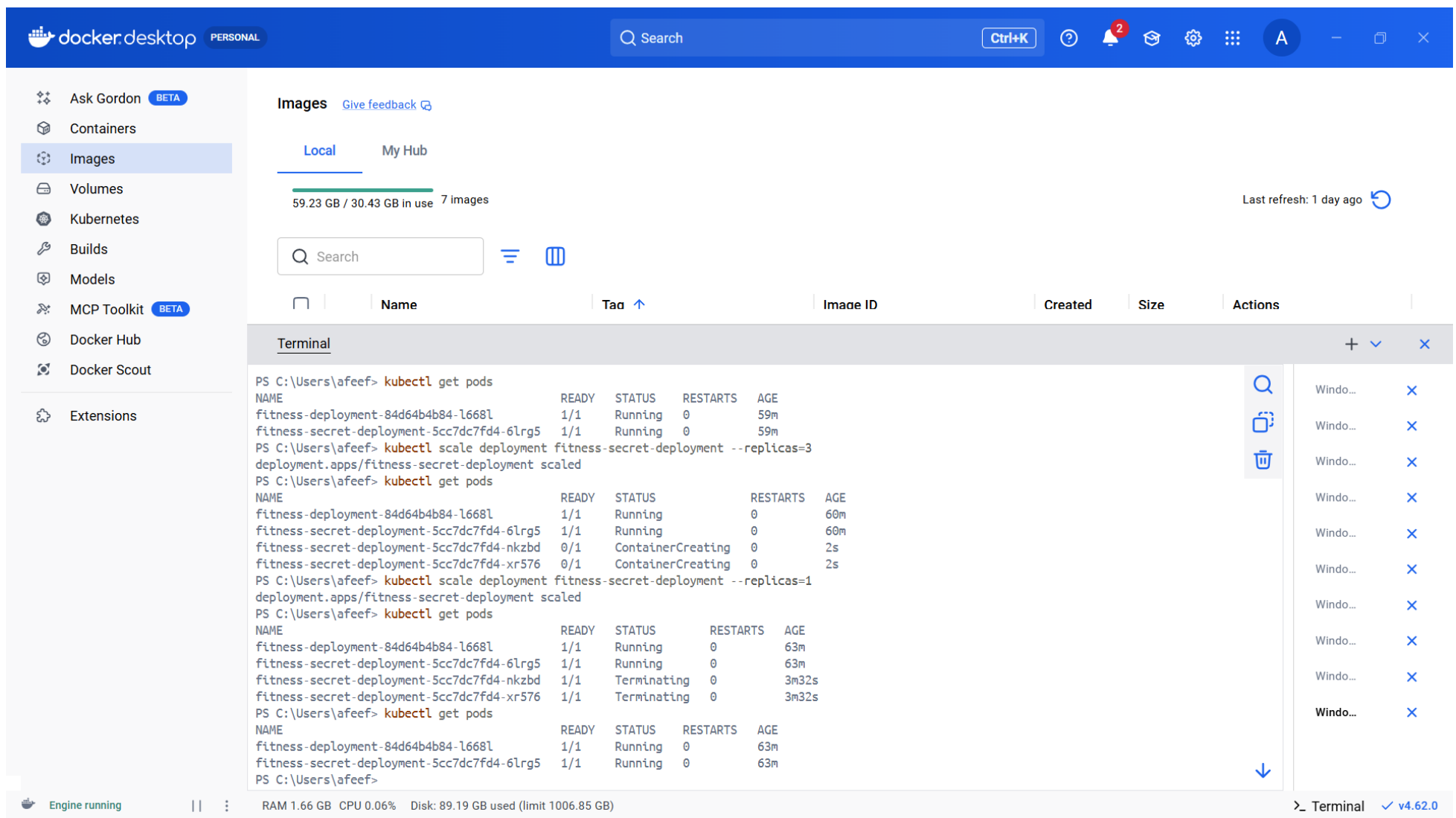


Figure 30: Downscaling

ALL FINAL DEPLOYMENT

After completing the deployment process, performing scaling operations, and integrating secrets, the final status of all deployments in the Minikube cluster was verified. This step ensures that all applications are correctly deployed and properly managed by Kubernetes.

To display all deployments in the cluster, the following command was executed:

```
kubectl get deployments
```

This command provides details such as:

- Deployment name
- Desired number of replicas
- Available replicas
- Current status

The output included deployments such as:

- **fitness-deployment**
- **fitness-secret-deployment**
- **hume-rainfall-deployment**

Each deployment showed the required number of replicas in the **READY** state, confirming that the applications were running successfully.

To obtain detailed information about a specific deployment, the following command can be used:

```
kubectl describe deployment fitness-secret-deployment
```

This command displays additional information, including:

- Pod template configuration
- Associated ReplicaSets
- Events related to scaling and updates

This final verification step confirmed that all deployments were successfully created, scaled, and managed within the Minikube Kubernetes cluster.

ALL PODS AT THE LAST

After completing all deployment, scaling, and secret configuration tasks, the final status of all active pods in the Minikube cluster was verified. This step ensures that the applications are operating correctly and that Kubernetes is managing them efficiently.

To list all running pods, the following command was executed:

```
kubectl get pods
```

This command displays details such as:

- Pod name
- Readiness status
- Current state
- Number of restarts
- Age of the pod

The output included pods associated with:

- **fitness-deployment**
- **fitness-secret-deployment**
- **hume-rainfall-deployment**

Each pod showed a status of **Running** with the required number of containers ready (for example, *1/1*), confirming that the applications were functioning properly within the cluster.

This final verification confirmed that all pods were successfully deployed, scaled, and securely configured, and were being properly managed by Kubernetes.

docker.desktop

PERSONAL

Q Search

Ctrl+K

?

2

A

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

61.22 GB / 30.43 GB in use 7 images

Last refresh: 2 days ago

Q Search

Name

Tao

Image ID

Created

Size

Actions

Terminal

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\afeef> kubectl get deployments

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
fitness-deployment	1/1	1	1	23h
fitness-secret-deployment	1/1	1	1	23h
hume-rainfall-deployment	1/1	1	1	4m44s

PS C:\Users\afeef>

RAM 1.55 GB CPU 1.00% Disk: 89.19 GB used (limit 1006.85 GB)

>_ Terminal

v4.62.0

Figure32: AllPods